



University of Asia Pacific

Department of Computer Science and Engineering

CSE 316: Microprocessors and Microcontrollers Lab

Code

Submitted by:

MD Faizullah Toushif(23101037)

Hamdil Hasan Partho(23101040)

Sumaya Akter Lima(23101041)

Shayla Hossen Simu(23101042)

Section : A2

Semester : 3rd year 2nd semester

Submitted to:

Khadiza Sultana Sayma

Lecturer, Department of Computer Science and Engineering

```

/*****
* Auto Plant Hydration System using ESP32
* Components:
* - HC-SR04 Ultrasonic (Water Level)
* - DHT11 Temperature + Humidity Sensor
* - Capacitive Soil Moisture Sensor (Analog)
* - Rain Sensor (Digital)
* - Servo Motor for Shed
* - 2-Channel Low-Trigger Relay (Pump Control)
* - Blynk IoT for Monitoring and Control
*****/

// ---- BLYNK SETTINGS ----
#define BLYNK_TEMPLATE_ID "TMPL6AoQp2_Md"
#define BLYNK_TEMPLATE_NAME "IOT Based Smart Hydration System"
#define BLYNK_AUTH_TOKEN "hqQ78-m2L7gerjlsCDf1BIgiGxR6c0LJ"
#define BLYNK_PRINT Serial
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <DHT.h>
#include <ESP32Servo.h>
// ---- WiFi credentials ----
char ssid[] = "UAP";
char pass[] = "abcelp@uap";
// ---- Pin Configuration ----
const int TRIG_PIN = 22;
const int ECHO_PIN = 23;
const int DHT_PIN = 4;
const int SOIL_PIN = 34;
const int RAIN_PIN = 15;
const int SERVO_PIN = 12;
const int RELAY_PIN = 21;
const int STATUS_LED = 2;
// ---- Threshold Values ----
const float TEMP_THRESHOLD = 30.0;
const float HUMIDITY_THRESHOLD = 40.0;
const int SOIL_THRESHOLD_RAW = 2000;
const float WATER_LEVEL_DISTANCE_THRESHOLD = 12.0;
const unsigned long PUMP_RUNTIME_MS = 10000UL;
// ---- DHT Setup ----
#define DHTTYPE DHT11
DHT dht(DHT_PIN, DHTTYPE);
// ---- Servo Setup ----
Servo shedServo;
const int SERVO_OPEN_ANGLE = 0;
const int SERVO_CLOSE_ANGLE = 90;
// ---- Blynk Virtual Pins ----
#define VP_TEMP V0
#define VP_SOIL V1
#define VP_WATER V2
#define VP_RAIN V3
#define VP_PUMP V4
#define VP_SERVO V5
#define VP_HUMIDITY V6

```

```

// ---- State Variables ----
bool pumpState = false;
bool pumpManualOverride = false;
bool pumpRunningByLogic = false;
bool manualServoControl = false;
bool tankSufficient = false;
unsigned long pumpStartMillis = 0;
BlynkTimer timer;
// ----- Ultrasonic Function -----
float readUltrasonicCM(int trigPin, int echoPin) {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    long duration = pulseIn(echoPin, HIGH, 30000);
    if (duration == 0) return -1.0;
    return duration / 58.0;
}
// ----- Pump Control -----
void pumpOn() {
    digitalWrite(RELAY_PIN, LOW);
    pumpState = true;
    pumpStartMillis = millis();
    pumpRunningByLogic = true;
    Serial.println(" Pump ON");
    Blynk.virtualWrite(VP_PUMP, 1);
    Blynk.logEvent("pump_started", "Pump started");
}
void pumpOff(const char* reason = "Stopped") {
    digitalWrite(RELAY_PIN, HIGH);
    pumpState = false;
    pumpRunningByLogic = false;
    Serial.print(" Pump OFF: "); Serial.println(reason);
    Blynk.virtualWrite(VP_PUMP, 0);
    Blynk.logEvent("pump_stopped", reason);
}
// ----- Blynk Control -----
BLYNK_WRITE(VP_PUMP) {
    int v = param.asInt();
    if (v == 1) {
        if (tankSufficient) {
            pumpManualOverride = true;
            pumpOn();
        } else {
            Serial.println("Blynk switch ON rejected - Tank too low");
            Blynk.virtualWrite(VP_PUMP, 0);
        }
    } else {
        pumpManualOverride = false;
        pumpOff("Manual Stop");
    }
}
}

```

```

BLYNK_WRITE(VP_SERVO) {
    int angle = param.asInt();
    shedServo.write(angle);
    manualServoControl = true;
}
// ----- Sensor Reading + Logic -----
void readSensorsAndAct() {
    float distance = readUltrasonicCM(TRIG_PIN, ECHO_PIN);

    // ---- DHT11 read with retry ----
    float temp = NAN;
    float humidity = NAN;
    for (int i = 0; i < 5; i++) {
        temp = dht.readTemperature();
        humidity = dht.readHumidity();
        if (!isnan(temp) && !isnan(humidity) && temp <= 60.0 && humidity <=
100.0) break;
        delay(1000);
    }
    if (isnan(temp) || temp > 60.0) {
        Serial.println("DHT11 ERROR - Check wiring!");
        temp = NAN;
        humidity = NAN;
    }

    int soilRaw = analogRead(SOIL_PIN);
    int rainDigital = digitalRead(RAIN_PIN);
    bool raining = (rainDigital == 0);
    // Debug prints
    Serial.print("Temp: "); Serial.print(temp);
    Serial.print(" C, Hum: "); Serial.print(humidity);
    Serial.print(" %, Soil: "); Serial.print(soilRaw);
    Serial.print(", Dist: "); Serial.print(distance);
    Serial.print(" cm, Rain: "); Serial.println(raining ? "YES" : "NO");
    // Send data to Blynk
    if (!isnan(temp) && temp <= 60.0) Blynk.virtualWrite(VP_TEMP, temp);
    if (!isnan(humidity) && humidity <= 100.0) Blynk.virtualWrite(VP_HUMIDITY,
humidity);
    Blynk.virtualWrite(VP_SOIL, soilRaw);
    if (distance > 0) Blynk.virtualWrite(VP_WATER, distance);
    Blynk.virtualWrite(VP_RAIN, raining ? 1 : 0);
    // Shed control
    if (!manualServoControl) {
        if (raining) shedServo.write(SERVO_CLOSE_ANGLE);
        else shedServo.write(SERVO_OPEN_ANGLE);
    }
    // Irrigation logic
    bool tankLow = (distance > WATER_LEVEL_DISTANCE_THRESHOLD && distance > 0);
    bool soilDry = (soilRaw > SOIL_THRESHOLD_RAW);
    bool tempHigh = (!isnan(temp) && temp <= 60.0 && temp >= TEMP_THRESHOLD);
    bool humidityLow = (!isnan(humidity) && humidity <= 100.0 && humidity <=
HUMIDITY_THRESHOLD);
    Serial.print("tankLow: "); Serial.print(tankLow);

```

```

Serial.print(", soilDry: "); Serial.print(soilDry);
Serial.print(", tempHigh: "); Serial.print(tempHigh);
Serial.print(", humidityLow: "); Serial.println(humidityLow);
// Pump control logic - manual switch only, no auto-start
if (tankLow) {
    tankSufficient = false;
    if (pumpState) pumpOff("Tank Low - Pump Disabled");
    Blynk.virtualWrite(VP_PUMP, 0);
    Serial.println("Tank LOW - Pump Disabled");
} else {
    tankSufficient = true;
    Serial.println("Tank OK - Pump Enabled (use Blynk switch to run)");
    Blynk.virtualWrite(VP_PUMP, pumpState ? 1 : 0);
}
// Pump runtime timeout
if (pumpState && millis() - pumpStartMillis >= PUMP_RUNTIME_MS) {
    pumpOff("Runtime Elapsed");
    pumpManualOverride = false;
}
digitalWrite(STATUS_LED, pumpState ? HIGH : LOW);
}
// ----- Setup -----
void setup() {
    Serial.begin(115200);
    pinMode(TRIG_PIN, OUTPUT);
    pinMode(ECHO_PIN, INPUT);
    pinMode(RAIN_PIN, INPUT_PULLUP);
    pinMode(RELAY_PIN, OUTPUT);
    pinMode(STATUS_LED, OUTPUT);
    digitalWrite(RELAY_PIN, HIGH);
    dht.begin();
    delay(2000); // DHT11 stabilization delay
    shedServo.attach(SERVO_PIN);
    shedServo.write(SERVO_OPEN_ANGLE);
    Serial.println("Connecting to WiFi...");
    WiFi.begin(ssid, pass);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\nWiFi connected!");
    Blynk.config(BLYNK_AUTH_TOKEN);
    Serial.println("WiFi connected, starting system...");
    timer.setInterval(5000L, readSensorsAndAct);
}
// ----- Loop -----
void loop() {
    Blynk.run();
    timer.run();
}

```